

Deliverable 4 Final Report

Locomotion Optimisation using Augmented Random Search and Cross-Entropy Method.

Optimisation Class Project - MuJoCo MJX / JAX

Julien Falandry

GitHub repository: <https://github.com/JulienFalandry/mujoco-ars-cem-locomotion>

Item	Value
Environment used	Walker2d locomotion proxy in MuJoCo MJX
CEM run	fairer_long: 80 iterations, population 384, elite fraction 0.10
ARS run	strong_stable: 280 iterations, 192 directions, top fraction 0.35
Best CEM return	1698.54
Best ARS return	5780.55
Best ARS iteration	277
ARS distance over 6 s	6.94 m
ARS mean forward velocity	1.16 m/s
ARS fall diagnostic	600 / 600 control steps

1. Introduction and objective

The objective of Deliverable 4 was to extend the optimisation pipeline from the earlier CartPole experiments to a more complex continuous-control locomotion problem. The assignment requested applying ARS to a high-dimensional locomotion environment, tuning its hyperparameters, comparing it with a failed or limited CEM attempt, and producing a final report, GitHub repository, and video rendering of the optimised gait.

The simulation was implemented with MuJoCo MJX, which allows the rollout evaluation to be vectorised across many candidate policies. This is important for both CEM and ARS, because both methods require evaluating many policies per optimisation iteration.

2. Implementation

2.1 Environment and policy representation

The locomotion task uses a Walker2d MuJoCo model. The model is transferred to MJX using `mjx.put_model`, and the rollout dynamics are executed through `mjx.step` inside JAX scan loops. Each control action is held for five MuJoCo physics substeps. With a MuJoCo timestep of 0.002 s, this corresponds to a control timestep of 0.01 s and a 600-step control horizon for a 6-second rollout.

The controller is a linear policy followed by a tanh activation, which keeps motor commands within the allowed range. The observation vector excludes the absolute x-position, so the policy observes body configuration and velocity, but not the global forward position. This avoids simply encoding position into the action and makes the task closer to learning a locomotion strategy.

2.2 Reward shaping

The reward was tuned to balance three behaviours: forward walking, upright stability, and moderate control effort. Earlier versions either rewarded standing too much or pushed the walker to sprint and fall. The final reward therefore keeps a positive forward-velocity term, penalises no-progress behaviour, adds pitch/height/control/smoothness costs, and includes a terminal bonus only when the walker survives and makes meaningful forward progress.

The final reward is not only a survival reward. This is important because a policy that simply stands upright can obtain a stable-looking video but does not solve the locomotion task. At the same time, the final configuration is more conservative than the fastest early run, because the goal of the final deliverable is a stable walking gait rather than maximum speed.

3. Optimisers

3.1 Cross-Entropy Method baseline

CEM samples a population of candidate policy vectors from a Gaussian distribution. At each iteration, all candidates are evaluated, the elite fraction is selected, and the distribution mean and standard deviation are refitted using the elite set. This implementation uses batched MJX rollouts for the population evaluation.

The final baseline uses the `fairer_long` mode with 80 iterations, a population of 384 policies, and an elite fraction of 0.10. This was done to avoid making the CEM baseline artificially weak. The extended CEM run used approximately 92.16 million physics steps and reached a best return of 1698.54.

3.2 Augmented Random Search

ARS evaluates pairs of positive and negative perturbations around the current policy parameters. It selects the best-performing perturbation directions and updates the policy in the weighted direction of improvement. The implementation follows the ARS V2 idea by maintaining running observation normalisation statistics.

The final ARS run uses the `strong_middle` configuration: 280 iterations, 192 perturbation directions, a top fraction of 0.35, step size 0.012, and exploration noise 0.035. Since each ARS iteration evaluates both positive and negative perturbations, this corresponds to 385 policy evaluations per iteration, including the updated policy evaluation. The final ARS run used approximately 323.4 million physics steps.

4. Results

4.1 Learning curves

Figure 1 compares the CEM attempt with the final ARS run. The CEM improves significantly from its initial negative returns and reaches a positive return, but it plateaus well below the best ARS result. ARS initially stays in a poor region, but after enough perturbation-based exploration it discovers a useful gait and climbs beyond the CEM baseline.

Part 4: Walker2d CEM attempt vs ARS tuning

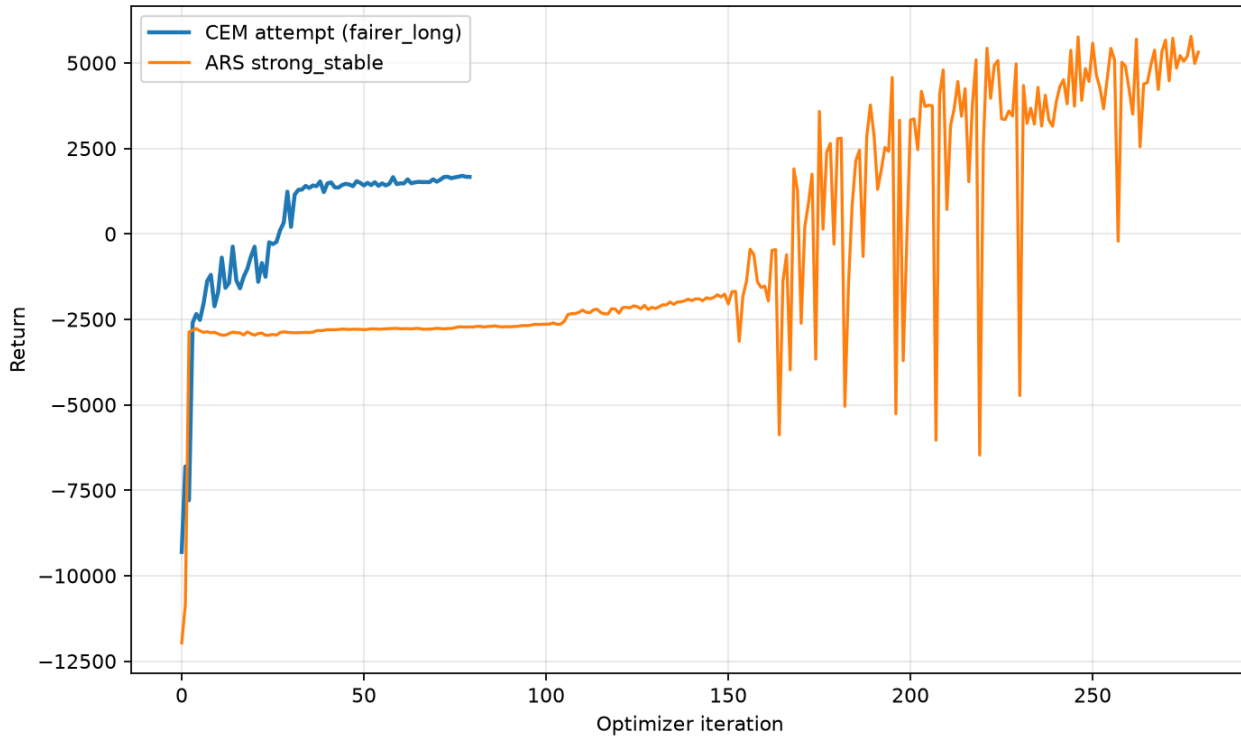


Figure 1. Learning curve for the extended CEM attempt and the final ARS tuning run on Walker2d.

Method	Run label	Iterations	Candidates per iter.	Elite/top fraction	α / v	Best return	Approx. physics steps
CEM	fairer_long	80	384	0.10	- / -	1698.54	92,160,000
ARS	strong_stable	280	2 x 192 + 1	0.35	0.012 / 0.035	5780.55	323,400,000

Table 1. Optimiser settings and sample budgets for the final comparison.

4.2 Final gait diagnostics

Metric	Value
Diagnostic return	5780.55
Distance travelled	6.94 m over 6 s
Mean forward velocity	1.16 m/s
Minimum torso height	0.937 m
Maximum absolute pitch	0.676 rad
First fallen step	600 / 600

Table 2. Final ARS diagnostic metrics over the 6-second horizon.

The final ARS policy travels 6.94 m in 6 seconds, corresponding to an average forward velocity of 1.16 m/s. It remains above the height threshold and below the pitch threshold for the full diagnostic horizon, with first

fallen step equal to 600/600. This indicates that the policy learned a stable forward gait rather than merely standing upright.

The CEM video was also rendered for the final long baseline. Visually, the CEM policy remains much more passive and does not exhibit the same clear forward-walking pattern as ARS. This supports the quantitative result: CEM improves with a larger budget, but it remains below ARS on this locomotion task.

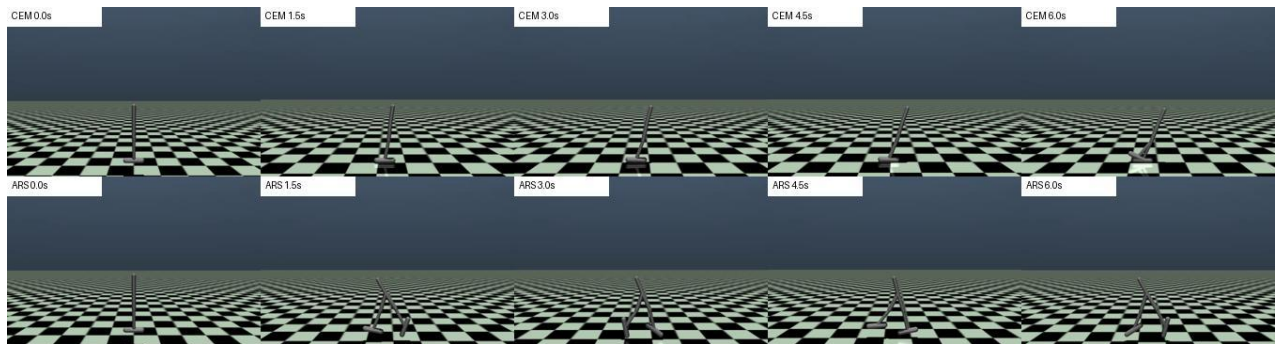


Figure 2. Video frame comparison: CEM policy (top row) and ARS policy (bottom row) over the 6-second rendering.

5. Discussion

5.1 Why ARS scaled better than CEM

CEM is simple and effective in lower-dimensional spaces, but it becomes difficult to scale when the policy has many parameters. It updates a sampling distribution from elite samples, so it needs a large population to estimate useful changes in the distribution reliably. In locomotion, many sampled policies fail or produce very noisy rewards, which makes the elite set unstable and slows convergence.

ARS uses a different strategy. It compares positive and negative perturbations along the same random directions, which gives a more informative estimate of whether moving in that direction improves the policy. It also uses the scale of elite rewards to normalise the update. This directional update is more suitable for high-dimensional linear policy search, especially when combined with state normalisation.

5.2 Sample efficiency interpretation

The final comparison should not be interpreted as a perfectly equal-budget benchmark. The extended CEM run used approximately 92.16 million physics steps, while ARS used approximately 323.4 million physics steps. However, the CEM baseline was extended to 80 iterations specifically to make the comparison more meaningful than the earlier 30-iteration attempt. Even with this larger budget, CEM reached a best return of 1698.54, whereas ARS reached 5780.55 and produced the final stable gait.

A stricter sample-efficiency study would run both optimisers under exactly the same total environment-step budget and repeat experiments over several random seeds. Within the scope of this project, the result still supports the main conclusion requested in the assignment: CEM struggles to scale to locomotion, while ARS is able to optimise a useful walking policy when given sufficient perturbation-based tuning.

5.3 Impact of JAX and MJX

JAX and MJX were essential because both optimisers evaluate large batches of rollouts. The implementation uses `jax.vmap` to evaluate candidate policies in parallel and `jax.scan` to compile the rollout

loop. On GPU, this allows hundreds of CEM candidates or ARS perturbations to be evaluated concurrently. Without MJX batching, the same experiments would be too slow for the available project time.

The main practical difficulty was GPU availability and compilation cost. Colab was not sufficient for the strongest experiments, so the training was moved to the University of Tartu HPC cluster. Once the environment was configured correctly, the GPU backend allowed the extended CEM run and the final ARS tuning run to complete and generate the final plots, metrics, and videos.

6. Limitations and possible improvements

The main limitation is reward sensitivity. The project showed that small reward changes can produce very different behaviour: standing still, unstable sprinting, or stable walking. Future work could include multiple random seeds, and a stricter equal-budget comparison between CEM and ARS.

7. Conclusion

The final experiment demonstrates that ARS can optimise a simple linear policy for a Walker2d locomotion task using batched MuJoCo MJX rollouts on GPU. The extended CEM baseline was given a more serious 80-iteration attempt and improved to a positive return, but it remained below the final ARS performance and did not produce the same quality of gait.

The best ARS policy reached a return of 5780.55, travelled 6.94 m over 6 seconds, and survived the full diagnostic horizon. This confirms that ARS was more suitable than CEM for this high-dimensional continuous-control setting. The use of JAX and MJX was critical for making the batched rollout evaluations feasible on GPU.